

Explicação da Lógica Técnica - Sprint 1

Elaborado por: Desenvolvedores da Sprint 1

Destinado a: Willian Gomes Pessoa (Quality Manager)

Sprint: 1 - Perfis e Autenticação (07/05 a 13/05/2026)

Visão geral da arquitetura

O backend é uma aplicação Flask organizada em **blueprints** - módulos independentes, um por domínio do sistema. A comunicação entre o browser e o servidor usa **sessão por cookie** (sem JWT). O banco de dados é MySQL, acessado por **queries SQL diretas** (sem ORM).

...

browser

- └─ Flask (app.py)
- └─ auth/ ← login, logout, primeiro acesso
- └─ usuarios/ ← CRUD de usuários, perfis
- └─ disciplinas/ ← (placeholder Sprint 1)
- └─ agenda/ ← (placeholder Sprint 1)
- └─ registros/ ← (placeholder Sprint 1)
- └─ relatorios/ ← (placeholder Sprint 1)
- └─ db/connection.py ← pool de conexões MySQL

...

TT01 - Stack

- Linguagem: Python 3.12 - familiaridade do time
 - Framework: Flask 3.0.3 - simples, sem "magia", cada rota é explícita
 - Banco: MySQL 8.0 - modelo relacional, SQL auditável
 - Conector: mysql-connector-python 9.0.0 - oficial da Oracle, sem dependências extras
 - Queries: SQL direto, sem ORM - transparência para revisão técnica e auditoria
-

TT02 - Modelagem do banco

O schema define 4 tabelas para suportar Sprint 1 e preparar Sprint 2:

`usuarios` - tabela central. Cada usuário tem um `papel` (ALUNO, MONITOR, PROFESSOR, ADMIN) e um `status` (PENDENTE, ATIVO, INATIVO). O campo `senha\_temporaria` indica se

o usuário ainda não trocou a senha inicial.

`password\_reset\_tokens` - tokens de reset de senha com expiração. Pertence a um usuário via FK com `ON DELETE CASCADE`.

`disciplinas` - tabela preparada para Sprint 2 (EP02). Já criada para evitar migration futura.

`monitorias` - vínculo entre aluno e disciplina. Constraint `UNIQUE(disciplina\_id, aluno\_id)` impede duplicidade.

TT03 - Setup local

O ambiente local usa **Docker Compose** para subir o MySQL sem instalar nada na máquina. O schema é aplicado automaticamente no primeiro start. O backend roda em **venv Python** com dependências fixadas em `requirements.txt`.

Fluxo de setup em 7 passos documentados em `docs/setup.md`:

1. Criar venv e instalar dependências
 2. Subir MySQL via Docker
 3. Configurar variáveis de ambiente (`env.example` como base)
 4. Criar o primeiro usuário ADMIN via script
 5. Rodar a aplicação
-

TT04 - Skeleton do projeto

A aplicação usa o padrão **app factory** (`create\_app()`): a instância Flask é criada dentro de uma função, o que facilita testes e configuração. Cada domínio é um **blueprint** registrado no factory.

A rota `GET /health` retorna `{"status": "ok"}` com HTTP 200 - usada para verificar se a aplicação está no ar.

Templates e arquivos estáticos (CSS, JS) estão em `frontend/`, separados do backend.

TT05 - Conexão com MySQL

A conexão usa **connection pool** com 5 conexões reutilizáveis - evita abrir e fechar conexão a cada request. Todas as configurações (host, porta, usuário, senha, banco) são lidas de **variáveis de ambiente** via a classe `Config`. Nenhuma credencial está hardcoded no código.

Se o banco não estiver disponível na inicialização, o erro é **logado no console** com `logging.exception` - a aplicação não sobe silenciosamente com banco ausente.

US01 - Cadastro de usuários com perfis

Fluxo principal (admin cria usuário):

1. Admin preenche nome, email e papel no formulário
2. Backend valida: papel deve ser ALUNO/MONITOR/PROFESSOR/ADMIN; nome e email obrigatórios; email não pode ser duplicado
3. Senha temporária é gerada com ``secrets.choice`` (10 caracteres alfanuméricos - criptograficamente seguro)
4. Hash da senha é armazenado no banco (`werkzeug `generate_password_hash``)
5. Usuário criado com status ``PENDENTE`` e ``senha_temporaria=True``
6. A senha temporária é exibida ao admin na tela - não é enviada por email

Outras operações disponíveis:

- Desativar usuário: atualiza status para INATIVO
- Reset de senha: gera nova senha temporária, exibe ao admin
- Monitor edita perfil: atualiza contato e disponibilidade no próprio perfil

US02 - Login com email e senha

Fluxo de login:

1. Usuário informa email e senha
2. Backend busca usuário por email e verifica com ``check_password_hash``
3. Se status for ``INATIVO``: rejeita com mensagem de conta inativa
4. Se status for ``PENDENTE``: redireciona para tela de **primeiro acesso** (troca de senha obrigatória)
5. Se status for ``ATIVO``: cria sessão com ``user_id``, ``nome``, ``papel`` e redireciona por papel:
 - ADMIN → gestão de usuários
 - MONITOR → perfil próprio
 - PROFESSOR/ALUNO → home

Primeiro acesso:

1. Usuário informa nova senha (mínimo 8 caracteres) e confirmação
2. Backend atualiza hash no banco e muda status para ``ATIVO``
3. Usuário é redirecionado ao dashboard

Segurança da sessão:

- Cookie com flag HTTPOnly (não acessível por JavaScript)
- Em produção, flag Secure ativado via variável de ambiente
- Sessão expira em 8 horas

US03 - Monitor edita seu perfil

Fluxo:

1. Monitor acessa `GET /usuarios/my-profile` - vê seus dados atuais
2. Preenche ou atualiza os campos editáveis e submete o formulário
3. Backend executa `UPDATE usuarios SET contato=?, disponibilidade=? WHERE id=?`
4. Redireciona de volta ao perfil com mensagem de sucesso

Campos editáveis:

- contato - email de contato ou telefone (texto livre)
- disponibilidade - descrição de horários livres (texto livre)

Controle de acesso:

- Decorador @login_required garante que apenas usuários autenticados acessam a rota
- O user_id vem da sessão do servidor - não pode ser falsificado pelo cliente
- Monitor só pode editar o próprio perfil (o id editado é sempre o da sessão)

Nota: A tela está intencional-mente simples nesta sprint. Ela será expandida nas sprints seguintes com exibição da agenda (US10) e das monitorias ativas (US07/US08).

US04 - Admin desativa um usuário

Fluxo:

1. Admin clica em "Desativar" na lista de usuários
2. Backend executa `POST /usuarios/<id>/desativar`
3. Serviço valida: admin não pode desativar a si mesmo
4. Atualiza `status = 'INATIVO'` no banco
5. A partir desse momento o usuário não consegue mais fazer login

Proteção contra lock-out:

- A validação `if user_id == current_user_id:` return error impede que o único admin desative a própria conta
- Essa regra está no `service.py` (camada de negócio), não no `template` - não pode ser contornada pela interface

Fluxo no login após desativação:

- `authenticate_user()` lê o status do banco antes de criar sessão
 - Se `status == 'INATIVO'`: retorna erro e não cria sessão - o usuário não entra no sistema
-

US05 - Admin reseta a senha de um usuário

Fluxo:

1. Admin clica em "Resetar senha" na lista de usuários
2. Backend executa `POST /usuarios/<id>/resetar-senha`
3. Nova senha temporária é gerada com `secrets.choice` (10 caracteres alfanuméricos - criptograficamente seguro)
4. Hash da nova senha é gravado no banco; `senha_temporaria = True`; `status = 'PENDENTE'`
5. A senha temporária é exibida ao admin na tela - não é enviada por email
6. Na próxima tentativa de login, o usuário é forçado a trocar a senha (fluxo de primeiro acesso)

Por que não usar SMTP:

Enviar email exigiria configuração de servidor SMTP, credenciais e tratamento de falhas - overhead desnecessário para o MVP. O admin comunica a senha ao usuário por outro canal (mensagem, presencialmente).

TT06 - Deploy na nuvem (Railway)

O que foi configurado:

- Procfile com gunicorn como servidor de produção (em vez do servidor de desenvolvimento do Flask)
- railpack.json declara provider: python - instrui o Railway a tratar o projeto como Python
- requirements.txt na raiz do projeto (Railway exige na raiz; o backend também tem o seu próprio)

Adaptações para o ambiente Railway:

O Railway usa nomes de variáveis diferentes dos convencionais para MySQL:

Variável padrão	Variável Railway
<code>DB_HOST</code>	<code>MYSQLHOST</code>
<code>DB_PORT</code>	<code>MYSQLPORT</code>
<code>DB_USER</code>	<code>MYSQLUSER</code>
<code>DB_PASSWORD</code>	<code>MYSQLPASSWORD</code>
<code>DB_NAME</code>	<code>MYSQLDATABASE</code>

O `config.py` foi atualizado para aceitar ambos os formatos com `os.getenv('DB_HOST')` or `os.getenv('MYSQLHOST')`.

Schema automático em produção:

O ``connection.py`` foi atualizado para aplicar o schema (``CREATE TABLE IF NOT EXISTS``) na primeira inicialização. Isso elimina a necessidade de executar migrations manualmente a cada deploy - o banco se auto-configura de forma idempotente.

Iterações necessárias:

O deploy exigiu 6 commits de ajuste. Os principais problemas foram:

1. Railway não detectava Python automaticamente - resolvido com ``railpack.json``
2. Variáveis de ambiente com nomes diferentes - resolvido com fallback no ``config.py``
3. Schema não era aplicado no primeiro boot em produção - resolvido com auto-apply no ``connection.py``

URL em produção: <https://web-production-1f724.up.railway.app>

Pontos de atenção para o QM

- Senha temporária: exibida uma única vez ao admin - não há recuperação posterior sem novo reset
- Email: normalizado para lowercase antes de salvar e comparar
- Status PENDENTE: usuário com este status não acessa o sistema - é forçado a trocar a senha primeiro
- Reset de senha (US05): recoloca o usuário em status PENDENTE - na próxima vez que entrar, terá que trocar a senha novamente
- Desativação (US04): irreversível pela interface - apenas um admin com acesso direto ao banco ou um novo endpoint (não implementado) poderia reativar
- Blueprints futuros: disciplinas, agenda, registros e relatorios existem como placeholders
 - sem rotas implementadas ainda